

TD VAT 3

Optimalité en FTP

La simulation, pour des systèmes de tâches indépendantes :

- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques ou sporadiques en monoprocesseur ;
- Est une CNS (Condition Nécessaire et Suffisante) d'ordonnançabilité pour les systèmes de tâches périodiques en multiprocesseur ;

Cependant, la simulation possède deux inconvénients majeurs :

- Dès lors qu'il existe des facteurs pratiques (exclusions mutuelles, parties non préemptibles, suspension, contraintes de précédence non consistantes avec les priorités), la simulation perd toute propriété de viabilité. Au mieux, elle peut être utilisée pour trouver un contre-exemple, ou bien une séquence hors-ligne à exécuter en-ligne, mais en aucun cas elle ne peut être utilisée pour valider un système. Elle devient dans ce cas une condition suffisante de non-ordonnançabilité (i.e. elle peut exposer un contre-exemple, mais ne l'expose pas forcément s'il existe).
- La simulation possède une complexité exponentielle, d'après (Chan & Norrish, 2016), $PPCM(1, 2, \dots, n+1) \geq 2^n$.

Plusieurs auteurs ont proposé des tests d'ordonnançabilité de complexité polynomiale. Cependant, on sait que la complexité du problème d'ordonnançabilité est Co-NP-difficile au sens fort s'il existe des dates de réveil d'une part (Leung & Whitehead, 1982), et que la complexité du problème de calcul de temps de réponse est NP-difficile au sens faible (pseudo-polynomial) même lorsque les tâches sont réveillées simultanément (Eisenbrand & Rothvos, 2008). Par conséquent, un algorithme de complexité polynomiale ne peut pas être exact. Les tests proposés sont donc généralement des heuristiques conservatives, en d'autres termes des conditions suffisantes (mais non nécessaires) d'ordonnançabilité. De plus, elles adressent une affectation de priorités particulière, en l'occurrence elles adressent les affectations de priorité optimales.

Rate Monotonic (RM), qui affecte les priorités de façon inversement proportionnelle à la période des tâches est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Liu & Layland, 1973) :

1. Toutes les tâches sont indépendantes préemptibles ;
2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à échéance sur requête (i.e. $D_i = T_i$).

Un algorithme d'ordonnancement A est **optimal** dans une classe d'algorithmes d'ordonnancement Ψ , si $A \in \Psi$ et si $\exists B \in \Psi$ tel que B ordonnance fiablement un système S, alors A ordonnance fiablement S.

Les deux tests polynomiaux les plus connus pour RM sont :

La **borne de Liu & Layland** (Liu & Layland, 1973) : un système de n tâches indépendantes à échéances sur requête est fiablement ordonné par RM sur un processeur si $U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(\sqrt[n]{2} - 1)$. De

plus, $\lim_{n \rightarrow \infty} (n(\sqrt[n]{2} - 1)) = \ln 2 \approx 0,693147$. Cela implique notamment qu'un système de tâches est ordonnançable (pour sûr) par RM si sa charge est inférieure ou égale à 69%.

La **borne hyperbolique** (Bini, Buttazzo, & Buttazzo, 2003) : soit la charge unitaire d'une tâche définie par $U_i = \frac{C_i}{T_i}$. Un système de n tâches indépendantes à échéances sur requête est fiablement ordonné par RM sur un processeur si $\prod_{i=1}^n (U_i + 1) \leq 2$. La borne hyperbolique est strictement plus puissante que le test de L&L, cela signifie que tout système validable par L&L est aussi validable par la borne hyperbolique, mais qu'il existe des systèmes validables par la borne hyperbolique qui ne passent pas le test de L&L. Cela rend théoriquement le test de L&L obsolète, cependant, comme on calcule toujours la charge d'un système pour vérifier qu'il n'est pas trivialement non ordonnançable, le test de L&L est immédiat, ce qui le rend tout de même intéressant à connaître.

Dans le cas où $D_i \leq T_i$, ces tests s'appliquent en remplaçant la charge U_i par la densité C_i/D_i . Ils deviennent cependant dans ce cas encore plus pessimistes que les tests à échéance sur requête.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonné en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	8	30	30
τ_2	0	6	20	20
τ_3	0	4	15	15

- Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
- RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
- Le système est-il démontré ordonnançable par un test de complexité polynomiale que vous connaissez ?
- Regarder l'exercice 1 dans le TD de simulation et conclure.

Exercice 2 :

Considérons un système de tâches différées, indépendantes et périodiques, ordonné en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	8	8,05	30	30
τ_2	4	4,5	20	20
τ_3	0	4,3	15	15

- Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
- RM donne-t-il une affectation de priorités fixes aux tâches optimale ?
- Peut-on tout de même appliquer le test de L&L et le test hyperbolique ? Si oui, les appliquer et conclure.

Deadline Monotonic (DM), qui affecte les priorités de façon inversement proportionnelle au délai critique est optimale dans la classe des algorithmes à priorités fixes aux tâches lorsque (Leung & Whitehead, 1982) :

- Toutes les tâches sont indépendantes préemptibles ;

2. Et le système est monoprocesseur ;
3. Et il existe un instant critique (i.e. les tâches peuvent être activées simultanément) ;
4. Et les tâches sont périodiques ou sporadiques (dans le cas sporadique, la condition 3 est forcément satisfaite) ;
5. Et Les tâches sont à **échéance contrainte** (i.e. $D_i \leq T_i$).

En fait, on peut voir RM comme le cas particulier de DM pour les tâches à échéances sur requêtes.

Exercice 3 :

Soit un système de tâches indépendantes, **sporadiques**, ordonnancé en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	WCET C_i	Délai critique D_i	Période T_i
τ_1	4	16	16
τ_2	3	7	8
τ_3	4	6	12

- a) Connaissez-vous une affectation de priorités fixes aux tâches optimale dans ce contexte ?
- b) Appliquer les tests de L&L et hyperbolique en considérant la densité, et conclure.
- c) Tracer une simulation du système suivant cette affectation de priorités.
- d) Existe-t-il une affectation de priorités fixes aux tâches ordonnant fiablement ce système ?

Exercice 4 :

Lorsqu'il existe des délais critiques plus grand que la période, ou bien lorsque certaines tâches sont différées et qu'il n'y a pas d'instant critique, RM et DM perdent leur optimalité. Il existe un moyen permettant de tester efficacement toute affectation de priorités possible, si le contexte satisfait 3 conditions :

- Condition 1: l'ordonnançabilité d'une tâche dépend des tâches plus prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 2: l'ordonnançabilité d'une tâche dépend des tâches moins prioritaires, individuellement, mais pas de leur ordre de priorités entre elles.
- Condition 3 : si une tâche respecte ses échéances avec un niveau de priorité i , elle ne peut pas ne pas les respecter avec un niveau de priorité $i+1$.

Si ces 3 conditions sont satisfaites par le test d'ordonnançabilité employé, alors on peut utiliser l'algorithme d'Audsley (Audsley, 1991), dit « Optimal Priority Assignment » ou OPA.

```

T=liste des tâches (triée par exemple par ordre croissant DM)
Pour priorité  $\Pi=1$  à  $n$ 
     $\tau$ =tete(T)
    Prio_affectée=faux
    Répéter
        Si  $\tau$  ne respecte pas ses échéances avec priorité  $\Pi$ 
            Si  $\tau$ =last(T) -- on a épuisé toute la liste
                Renvoyer non ordonnançable
            Sinon  $\tau$ =next(T)
        Sinon
            Prio_affectée=vrai
            Assigner priorité  $\Pi$  à  $\tau$  et remove(T, $\tau$ )
    Jusqu'à Prio_affectée
Fait

```

- a) Quelle est la complexité de cet algorithme en supposant que vérifier qu'une tâche respecte ses échéances ait une complexité $O(1)$?
- b) Supposons que nous ayons 25 tâches, combien au maximum de test d'ordonnançabilité de tâche devons-nous mener ?

Une façon de tester si une tâche respecte ses échéances, indépendamment des autres priorités, consiste à simuler (si on se trouve dans un contexte où la simulation est viable) le système en affectant :

- La priorité Π à τ
- La priorité $\Pi+1$ à toutes les autres tâches encore présentes dans T
- La priorité déjà choisie pour les tâches affectées précédemment

On ne regarde alors que pour la tâche τ considérée si les échéances sont respectées (on ne regarde pas si les échéances des tâches plus prioritaires sont respectées ou non).

- c) En supposant que le test d'ordonnançabilité de tâche retenu soit la simulation, est-ce que, si les tâches sont non préemptibles, on peut utiliser OPA ?
- d) Observer le système de tâches sporadiques indépendantes et préemptibles suivant. DM est-il optimal dans ce cadre ? OPA s'applique-t-il en considérant la simulation pour valider l'ordonnançabilité des tâches ? Le système est-il ordonnançable en priorités fixes aux tâches ?

	WCET C_i	Délai critique D_i	Période T_i
τ_1	4	16	16
τ_2	2	9	8
τ_3	5	13	12

Bibliographie

- Audsley, N. C. (1991). *Optimal priority assignment and feasibility of static priority tasks with arbitrary start times*. Computer Science Report YCS 164, York.
- Bini, E., Buttazzo, G. C., & Buttazzo, G. M. (2003). Rate monotonic analysis: the hyperbolic bound. *IEEE Transactions on Computers* 52(7), 933-942.
- Chan, H.-L., & Norrish, M. (2016). Proof pearl: bounding least common multiples with triangles. *International Conference on Interactive Theorem Proving*, (pp. 140-150).
- Eisenbrand, F., & Rothvoss, T. (2008). Static-priority real-time scheduling: Response time computation is NP-hard. *Real-Time Systems Symposium*, (pp. 397-406).
- Leung, J. Y.-T., & Whitehead, J. (1982). On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance evaluation*, 237-250.
- Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM* 20 (1), 46-61.